DEEP LEARNING PROJECT

Master in Data Science and Advanced Analytics

NOVA Information Management School

Universidade Nova de Lisboa

Family Cassification

Group 07

Fábio dos Santos, 20240678

Joana Rodrigues, 20240603

Mara Simões, 20240326

Matilde Domingues, 20240523

Rafael Borges, 20240497

Table of Content

1. Introduction 2
2. Exploration of dataset 2
3. Models 3
3.1. From scratch 3
3.1.1. Preprocessing 3
3.1.2. Composition 4
3.2. Pre-trained 5
3.2.1 Backbone selection 5
3.2.2 End-to-end pipeline 5
3.2.3 Regularization & Choises 5
3.2.4 Sweep Configuration 5
3.2.5 Data Augmentation Tuning 6
4. Conclusion 6
5. Future Work 6
5. References: 7
6. Annex A – Metrics used 8
7. Annex B - Figures 8
8. Annex C - Tables 13

1. Introduction

The Encyclopedia of Life (EOL) is an open-access database with images and metadata of species from all over the world [1]. The dataset for this deep learning project contains images of rare species from that website and their corresponding metadata. This way, the goal is to develop a model that is able to predict the family of the species that it is presented with using deep learning techniques. This is important because it allows the automation of species classification leading to faster and more efficient processes, especially valuable in large datasets, as applied by Norouzzadeh et al. (2019), that used deep learning to classify animal species in camera trap images [2].

The dataset contains images from 202 different families, each with different quantities of images. To address this, two approaches were explored:

1. Building a convolutional neural network (CNN) from **scratch**, trained entirely on the preprocessed image data.
2. Using different **pretrained models** through transfer learning and tuning their parameters.

The main challenge found was to create a model capable of identifying all the 202 different types of biological families, since the dataset was very imbalanced. Following the research made by Buda et al. (2018) that showed that oversampling minority classes helps reduce imbalance without overfitting [3], our expectation was that by using these techniques the model's performance would improve, which was not entirely the case. Furthermore, in a paper Dodge and Karam (2016) highlighted the importance of image quality in a deep learning model to improve the model's performance [4]. Therefore, removing low quality images (noise and blur) and improving the image quality of the remaining became a priority in this project.

As a result, several libraries were used, mainly TensorFlow and Keras to build, train, and evaluate the models and Weights & Biases (WandB) to track performance scores.

2. Exploration of dataset

To understand the data, an overview of the metadata was made. Firstly, the distribution of the classes (family and phylum) was analyzed (Figure 1). This way, 202 different phylum classes were identified, with the smallest class containing 29 images and the largest up to 300. Furthermore, it was seen that *Chordata phylum* dominated the dataset (Figure 2).

The images' composition was explored to guarantee that they were in a compatible format, as RGB is the most common image mode (Figure 3) and they are mostly squared (Figure 4), the model will be developed with that format. Finally, as the dataset contained several black and white images, these were seen to understand whether they were related to errors or not.

3. Models

Experimental Setup

Initially, experiments were conducted manually. However, we quickly recognized the need for a tool to help track models, results, and our various attempts more efficiently.

To address this, we integrated Weights & Biases (W&B) into our workflow which recorded every run, displayed the evolving metrics in real time and coordinated the entire machine-learning workflow. Hyper-parameter searches were launched through W&B's sweep agent: the advantage is that every tunable value lives in a single *sweep_config* dictionary, starting a new exploration required nothing more than editing that dictionary. We chose Bayesian optimization as the search strategy, since it proposes each new configuration based on previous results and therefore converges with fewer training runs than either grid or random search [5].

Training is compute-intensive, so we decided to run code on Google Colab. While Colab provides free GPUs, its 12-hour session limit often interrupts sweeps and impairs the Bayesian optimizer's ability to learn from prior runs.

Because the dataset is imbalanced class-specific weights were supplied to the loss function—categorical or focal cross-entropy—ensuring that under-represented labels influence loss function as strongly as the majority class.

3.1. From scratch

3.1.1. Preprocessing

To ensure the images and its metadata are prepared for the model, several preprocessing steps were taken to improve the model's ability to predict [6]. Firstly, to avoid data leakage, the dataset was split into train (60%), validation (20%), and test (20%) sets and the class names were encoded.

To guarantee the model receives correct images so it isn't biased, three different approaches were tested for the removal of images that were considered outliers. The approach that performed best was chosen based on the f1 score obtained from a sequential model. The different tries are as follows:

1. Extracting image features using the VGG16 pre-trained model and using HDBSCAN to group similar images together, while treating images that don't belong to any group as outliers.
2. VGG16 to extract features and Isolation Forest for anomaly detection (chosen for its speed and efficiency on large datasets).
3. Using clip model to calculate the cosine distance of images from the center of the group and removing the ones with a distance higher than 0.3 for classes with more than 50 images and 0.4 for the others. This approach was chosen based on the findings of Wang, S., Cheng, N., & Hu, Y. (2025), who demonstrated that the CLIP model improves detection accuracy and anomaly identification [7]. This was the best approach (Table 1).

A function was created to handle several preprocessing steps. The function does a central crop (keeping the central 80% of the images) to focus on the relevant areas. After cropping, the image is resized to the target dimensions (224x224 pixels). If the image is smaller than the chosen dimensions, padding is added around the image. Using the rescaling layer, the function normalizes the pixel values by scaling them to [0, 1]. Finally, labels are transformed using the one-hot encoding to make them suitable for multi-class classification.

As for techniques to fight the class imbalance, two were tried:

1. Data augmentation implemented to the model (in all classes) and the creation of a dataset with the images augmented proportionally to their class size (with high changes to guarantee that the images were a bit different from the original to prevent overfit).
2. Class weights so that the model could give more importance to the minority classes.

While class weights improved the f1 score, data augmentation decreased significantly (Table 2).

3.1.2. Composition

Convolutional neural networks from scratch using both the Sequential and Functional APIs in Keras were used in this section. The metrics used to evaluate the models were chosen considering the problem at hand: a multiclass classification with high class imbalance (Annex A).

Sequential Model

The baseline of the sequential model was a simple convolutional neural network with:

- Two convolutional layers with ReLU activation functions followed by a MaxPooling2D layer.
- A Flatten layer and a final Dense output layer with softmax activation function.

To achieve better results, a Bayesian search was applied to optimize the model's hyperparameters. From this optimization, the best model was achieved with the original architecture but replaced ReLU with LeakyReLU activation and used the RMSprop optimizer with a tuned learning rate. This configuration increased the F1 score (indicating better performance), but it also increased the loss (indicating an increase in incorrect predictions).

Functional Model

The baseline of the functional model was trained with the following composition:

- Two convolutional blocks each with Conv2D, MaxPooling2D, and LeakyReLU activation function.
- A Flatten layer and a final dense output layer with a softmax activation function.

After using the Bayesian search, the best configuration was composed of two Conv2D layers (with 64 and 32 filters), LeakyReLU activation (with alpha=0.48), MaxPooling, Flatten, and a Dense softmax output layer. This model was also tested with additional layers, but the scores did not improve.

To improve performance, we compared different versions of our models before and after applying Bayesian search (Table 2). We can conclude that both models showed improvements in accuracy and F1 score after tuning. However, the loss increased in the functional model after optimization. The best F1

score was achieved by the Sequential model with Bayesian search, showing a great balance between accuracy and loss.

3.2. Pre-trained

3.2.1 Backbone selection

Four ImageNet-pre-trained convolutional networks were benchmarked: Xception, ResNet-50 V2, VGG16 and EfficientNet. Because the first two models accept 299 × 299 inputs, every image is resized to 299 pixels on its shorter side during load with *image_data_from_directory.* VGG16 and EfficientNet were excluded early in the project for further optimization due to a big gap of performance in initial tests when comparing to the other two models (Table 3).

3.2.2 End-to-end pipeline

Each network begins with an input layer and a function that applies the selected data-augmentation, followed by a rescaling layer that maps pixel values to the range [-1, 1]. The chosen backbone—either Xception or ResNet-50 V2—comes next, after which the representation is fed through a global-average-pooling layer, and a fully connected block immediately followed by batch normalization, a dropout layer and, finally, a soft-max classifier.

3.2.3 Regularization & Choises

To minimize overfitting, the architecture exposes both dropout probabilities and batch normalization parameters to the optimizer. ReLU activations are paired with He-normal initialization, as recommended by Datta (2020), since this combination has consistently been shown to accelerate convergence in deep networks [8]. Additionally, we omit bias terms in dense layers to avoid redundancy, as batch normalization will re-center the activations in the subsequent step, following the approach proposed by Ioffe and Szegedy (2015). To further stabilize training, we implemented an early stopping callback—with a patience of 5 for short runs and 10 for long runs—and a learning rate reduction on plateau, triggered after 7 epochs without improvement in long runs [9].

3.2.4 Sweep Configuration

During the sweeps, we varied several hyperparameters, including batch size, learning rate, dropout rate, label smoothing, focal loss parameters, the number of units in the dense layer, the choice and strength of data augmentation, the optimizer, the backbone model, and the loss function. The validation macro F1-score was used as the optimization objective. Table 4 outlines the search space for our initial approach. A total of 19 runs were conducted. The overall results are presented in Figure 5, Figure 6 and Table 5. Based on these results, we selected four top-performing candidates (Figure 7). To choose the final model, we calculated a metric illustrated in Figure 8, which measures the performance gain relative to the gap between training and validation scores — with lower values indicating better generalization. According to this metric, the ‘happy-sweep-3’ run (green line) was selected as our baseline model, with its hyperparameters detailed in Table 6, even though we had runs with better scores such as 'easy-sweep-5' with peak validation F1-score of 61% we preferred to go on with a less overfitted run.

3.2.5 Data Augmentation Tuning

Figure 9 reveals a clear pattern: every top-performing run was trained without data augmentation. That insight prompted us to revisit our augmentation pipeline and launch a grid search to identify which—if any—augmentation combinations boost performance. The augmentation tested is in Table 7. In total we managed to run 34 out of the 64 possible combinations and our best balance between validation F1-Score and overfitting was the 'fancy-sweep-12' that used only random rotation, zoom and flip augmentation layers.

4. Conclusion

As a final attempt to improve performance, we conducted extended training runs using the best configurations identified—namely *fancy-sweep-12-longrun* and *happy-sweep-3-longrun*. These runs had no preset limit on the number of epochs and were allowed to continue until stopped by the early stopping callback.

The best results were achieved by the original 'happy-sweep-3', that considering the epochs needed to train it hold similar results to the other competitors. With a macro F1-score of 60.22% on the test set. In contrast, the scratch model reached a maximum F1-score of 4.38%, highlighting a significant performance gap compared to pretrained models. Despite efforts in normalization and data augmentation, the scratch model still exhibited signs of overfitting, with a train-validation F1-score gap of more than 40%. Bayesian search was done targeting the configuration that maximizes the F1 score for the validation set. Perhaps that contributed to overfitting and further experiments targeting loss should be executed.

5. Future Work

A few ideas were left unexplored, these include experimenting with pretrained models hosted on platforms such as Hugging Face, exploiting phylum-level metadata to implement hierarchical classification, and selectively unfreezing the upper layers of the base network for fine-tuning with a lower learning rate. Our review of the literature suggests that pursuing these directions would meaningfully advance and extend this project.

5. References:

1. Stevens, S., et al. (2024). *BioCLIP: A vision foundation model for the tree of life*. CVPR.
2. Willi, M., Pitman, R. T., Cardoso, A. W., Locke, C., Swanson, A., Boyer, A., ... & Fortson, L. (2019). Identifying animal species in camera trap images using deep learning and citizen science. *Methods in Ecology and Evolution*, *10*(1), 80–91. https://doi.org/10.1111/2041-210X.13120
3. Buda, M., Maki, A., & Mazurowski, M. A. (2018). A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks, 106*, 249–259. https://doi.org/10.1016/j.neunet.2018.07.011
4. Dodge, S., & Karam, L. (2016). Understanding how image quality affects deep neural networks. *arXiv preprint arXiv:1604.04004*. https://arxiv.org/abs/1604.04004
5. Yousif, M. J. (2023). Enhancing the accuracy of image classification using deep learning and preprocessing methods. *Artificial Intelligence & Robotics Development Journal*.
6. Weights & Biases. (2021). *What is Bayesian Hyperparameter Optimization? (With Tutorial)*. Retrieved from https://wandb.ai/wandb_fc/articles/reports/What-Is-Bayesian-Hyperparameter-Optimization-With-Tutorial---Vmlldzo1NDQyNzcw
7. Wang, S., Cheng, N., & Hu, Y. (2025). Comprehensive Environmental Monitoring System for Industrial and Mining Enterprises Using Multimodal Deep Learning and CLIP Model. IEEE Access. https://ieeexplore.ieee.org/abstract/document/10852209
8. Datta, L. (2020). A Survey on Activation Functions and their relation with Xavier and He Normal Initialization. arXiv preprint arXiv:2004.06632. https://arxiv.org/abs/2004.06632
9. Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. *arXiv preprint* arXiv:1502.03167. https://arxiv.org/abs/1502.03167

6. Annex A – Metrics used

Metric	Reason
Accuracy	Measures the number of species correctly identified.
F1 Score	Balances precision and recall. This metric is useful for our imbalanced dataset.
Loss	Represents how far the predictions of the model are from the true labels (a lower loss indicates better learning) - good to see if our model was overfitting.

7. Annex B - Figures

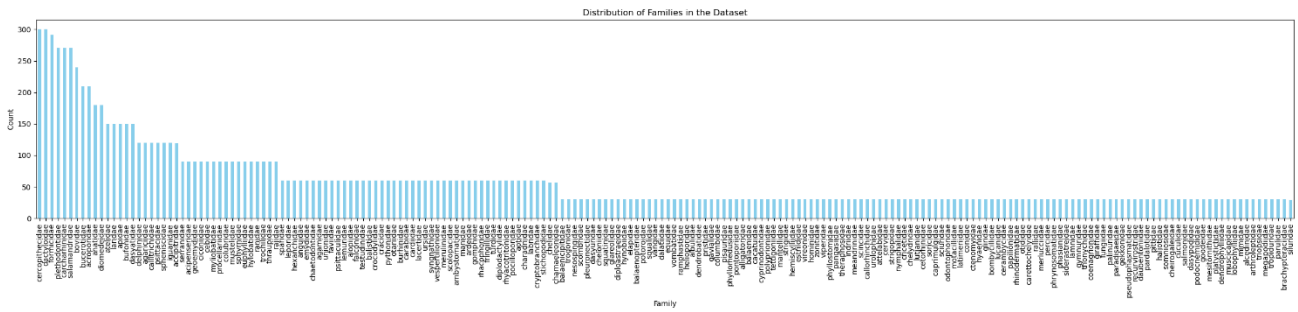

Figure 1 - Distribution of families in the dataset

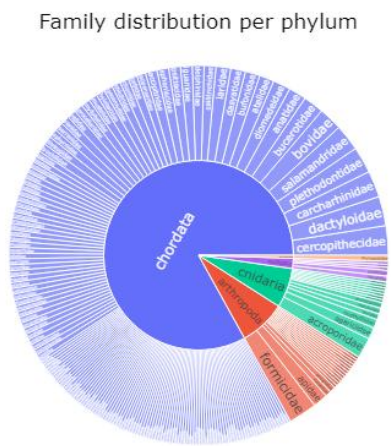

Figure 2 - Family distribution per phylum

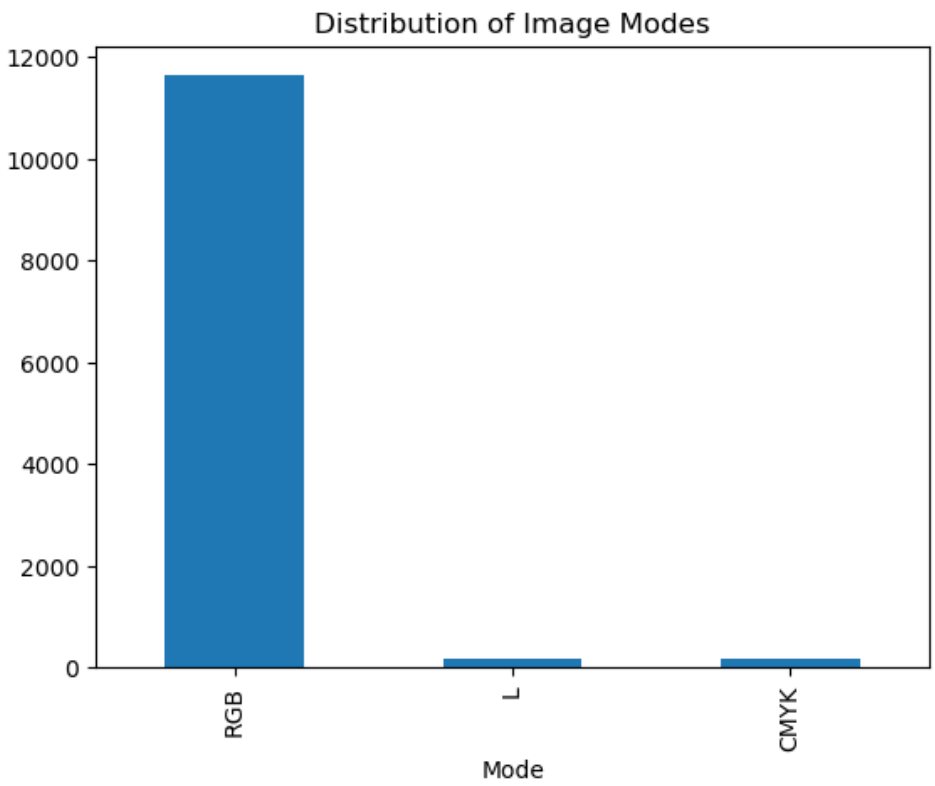

Figure 3 - Distribution of Image Modes

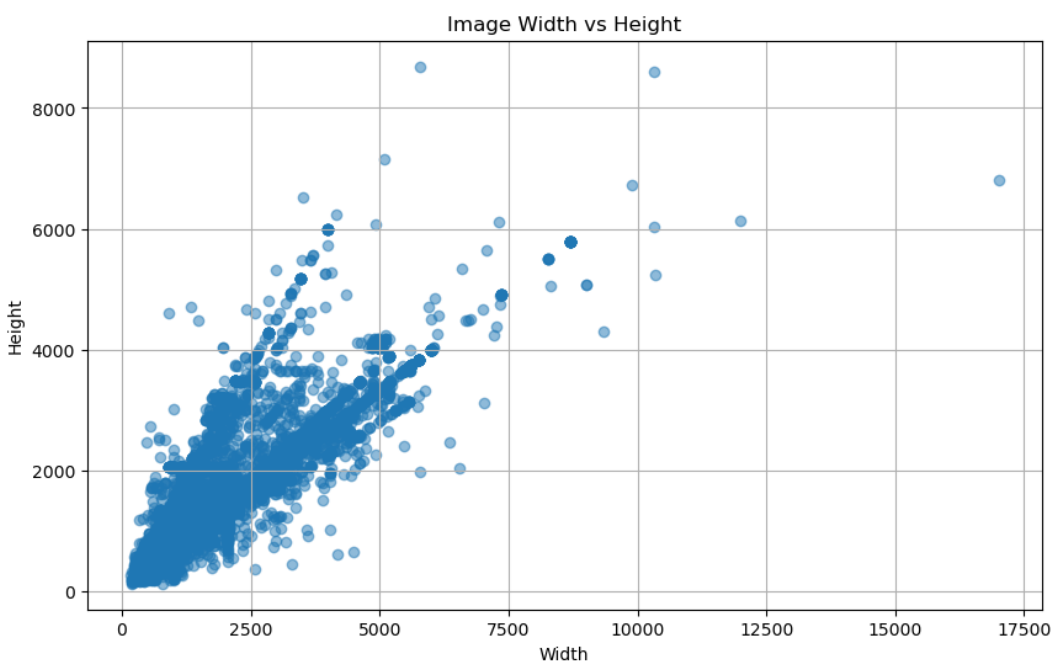

Figure 4 - Image Width and Image Height

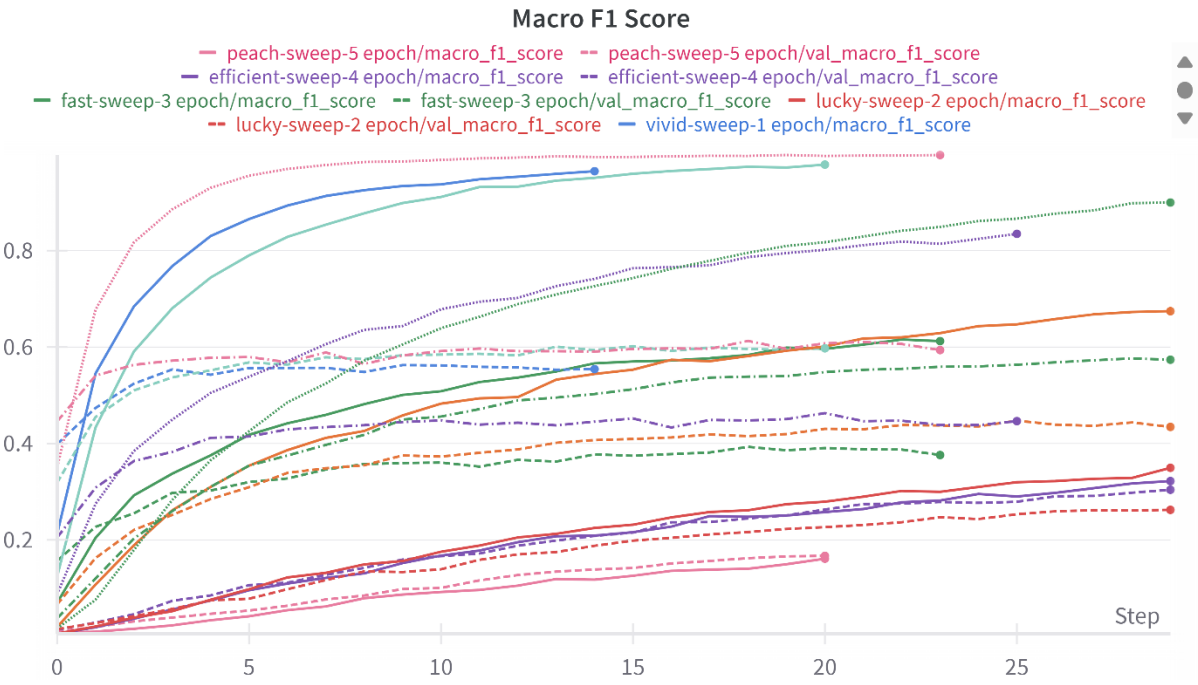

Figure 5 - Search Space Runs Macro F1 Score

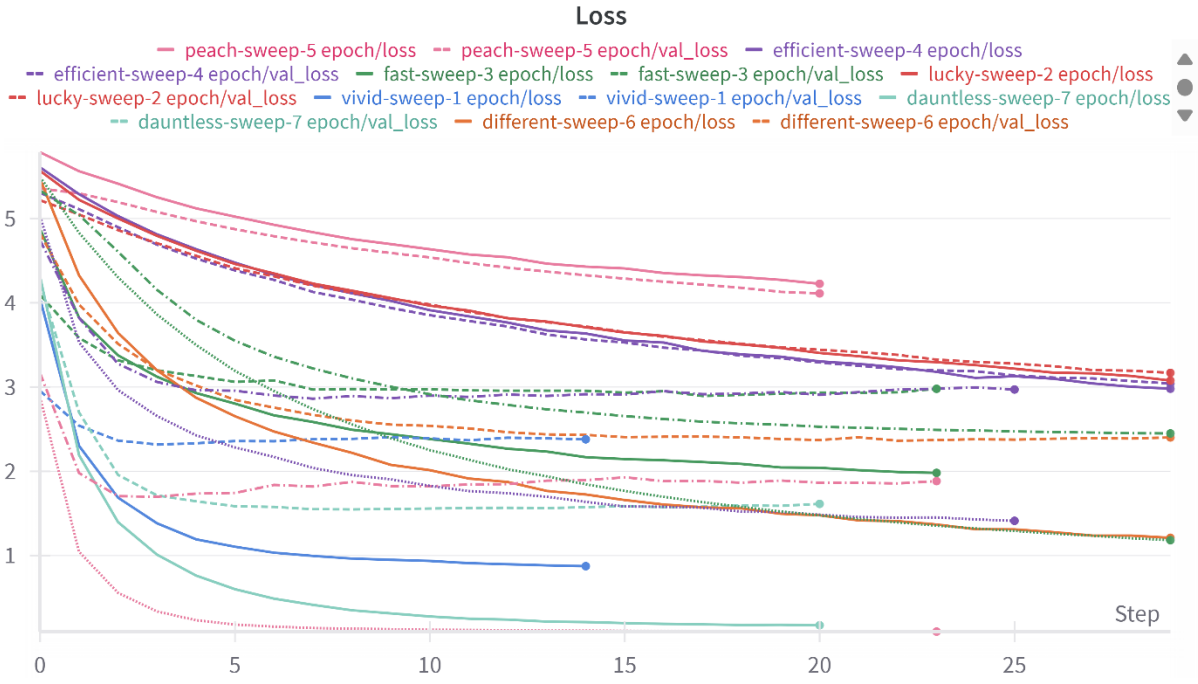

Figure 6 - Search Space Losses

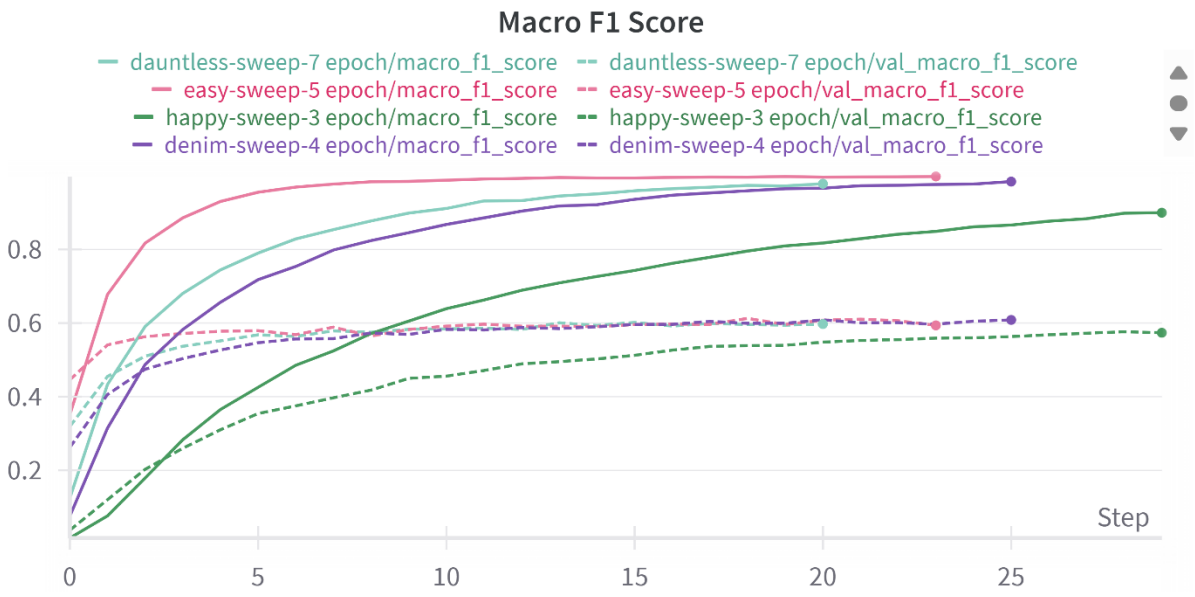

Figure 7 - Potential Best Runs

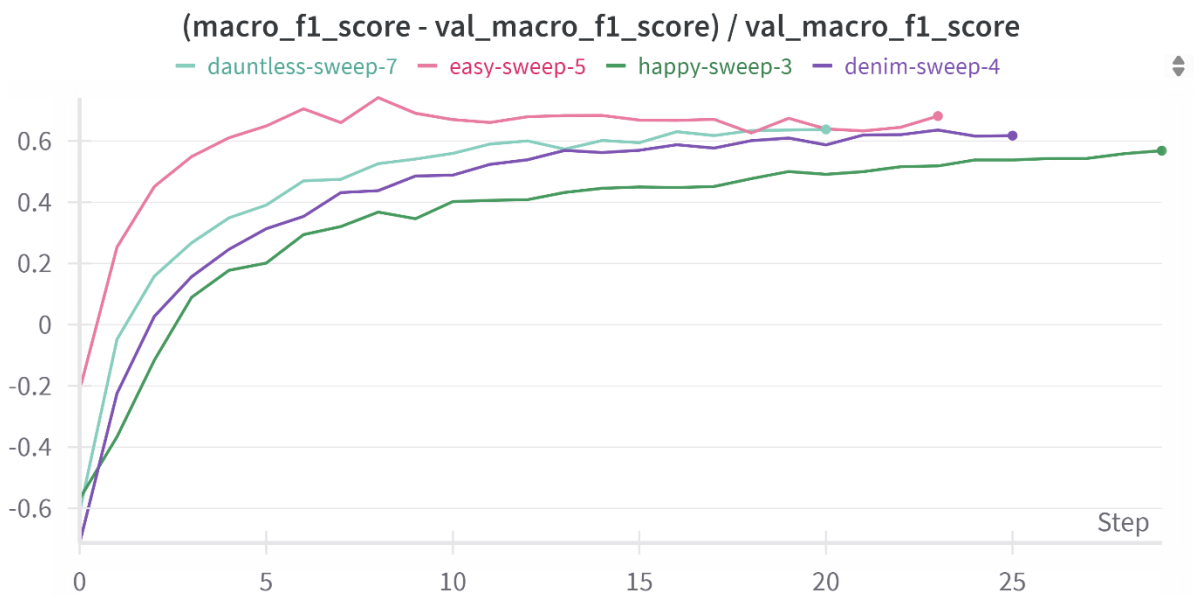

Figure 8 - Metric for Potential Best Runs

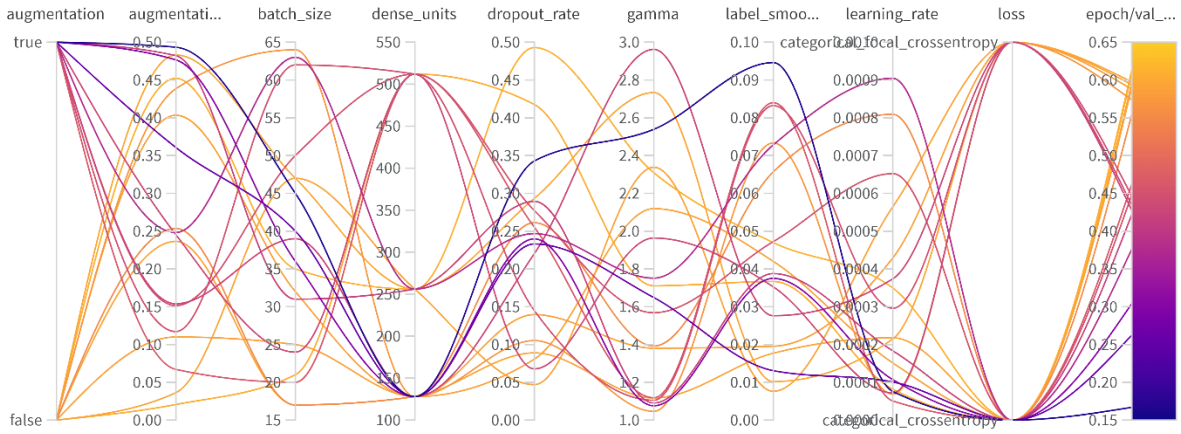

Figure 9 - Parallel Coordinate Chart

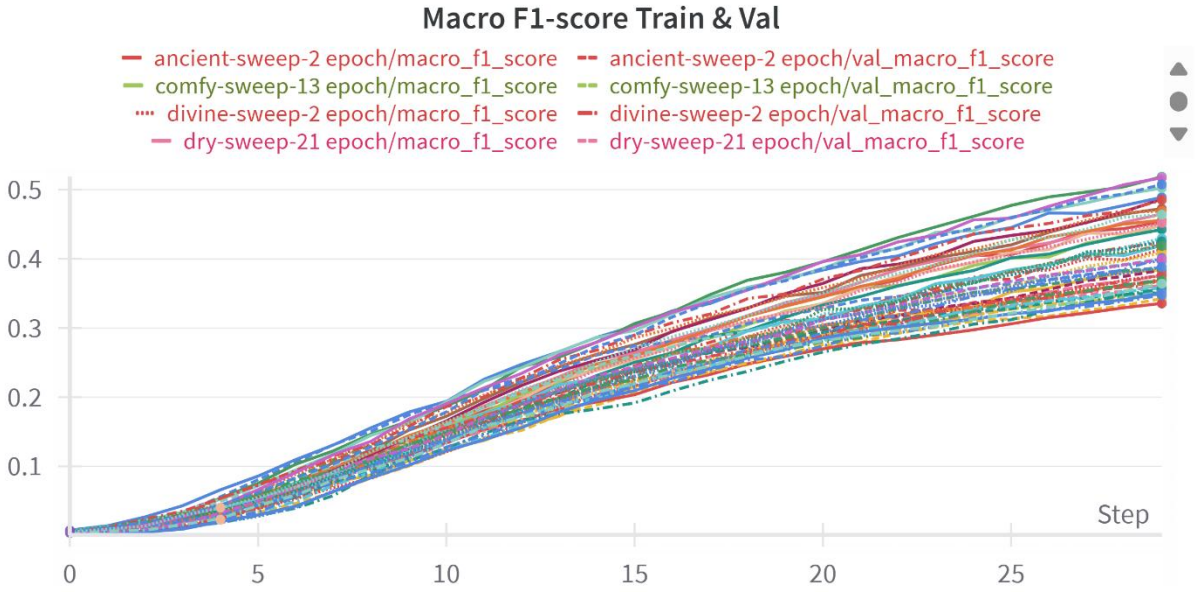

Figure 10 - All 34 augmentation runs

8. Annex C - Tables

Table 1 - Outlier Removal Comparison

Method	Accuracy	F1 score	Loss
With all images	0.026	0.021	14.438
DBSCAN	0.049	0.040	26.314
Isolation Forest	0.064	0.041	27.702
CLIP	0.065	0.047	24.468

Table 2 - Scratch Model Comparison

Model	epochs	Accuracy	F1 score	Loss
Sequential - initial	**10**	0.0317	0.0207	8.8516
Sequential - bayesian search	**10**	0.0317	0.0319	22.7174
Sequential - without class weights	**10**	0.0246	0.0215	21.2632
Sequential - dataset augmented	**10**	0.0062	0.0065	19.5562
Best sequential	**50**	0.0421	0.0364	179.1308
Functional - initial	**10**	0.0346	0.0287	13.0643
Functional - bayesian search	**10**	0.0354	0.0350	7.7662
Best functional	**50**	0.0516	0.0438	20.8757

Table 3 - Pre-trained models tests not recorded in Weights and Biases

Model	Version	Accuracy	F1 score	Loss	Auc
EfficientNet	Default without W&B	0.0371	0.0022	5.0323	0.4982
EfficientNet	With enhanced data augmentation without	0.0037	0.0023	6.2858	0.4953
VGG16	Default without W&B	0.5538	0.4986	0.4732	0.9185
VGG16	With enhanced augmentation, custom	0.5781	0.5314	0.4556	0.9314
VGG16	Same as above but without outliers	0.4946	0.4390	0.5630	0.9089
VGG16	Using sweep and W&B	0.0150	0.0080	5.9918	N/A

Table 4 - Search Space

Parameter	Distribution / Type	Min	Max	Values / Choices
batch_size	int_uniform	16	64	–
learning_rate	log_uniform_values	0.00005	0.001	–
dropout_rate	uniform	0.0	0.5	–
label_smoothing	uniform	0.0	0.1	–
gamma	uniform	1.0	3.0	–
dense_units	values (discrete)	–	–	512, 256, 128
augmentation	values (boolean)	–	–	True, False
augmentation_factor	uniform	0.0	0.5	–
optimizer	values (categorical)	–	–	adam, RMSprop
model	values (categorical)	–	–	xception, ResNet50V2
loss	values (categorical)	–	–	categorical_crossentropy, categorical_focal_crossentropy

Table 5 - Mentioned Runs

Sweep Name	Macro F1-Score	Val Macro F1 - Score	Test Macro F1- Score	Epochs Trained
Happy-sweep-3	0.8984	0.5764	0.6022	30
Happy-sweep-3-longrun	0.997	0.570	0.6022	88
Easy-sweep-5	0.9964	0.6127	-	23
Fancy-sweep-12	0.4490	0.3881	-	30
Fancy-sweep-12-longrun	0.8938	0.5704	0.60558	223

Table 6 - Happy Sweep Run Hyperparameters – Baseline Model

Hyperparameter	Value
Model	xception
Batch Size	64
Dropout Rate	0.10521598843788782
Label Smoothing	0.07333693134130108
Gamma	1.0461139526339738
Dense Units	128
Augmentation	False
Optimizer	adam
Loss	categorical_focal_crossentropy
Learning Rate	6.859458235313327e-05

Table 7 - Augmentation Search Space

Augmentation Type	Values Tested
Translation	True, False
Rotation	True, False
Zoom	True, False
Contrast	True, False
Saturation	True, False
Brightness	True, False
Flip	True (fixed value)